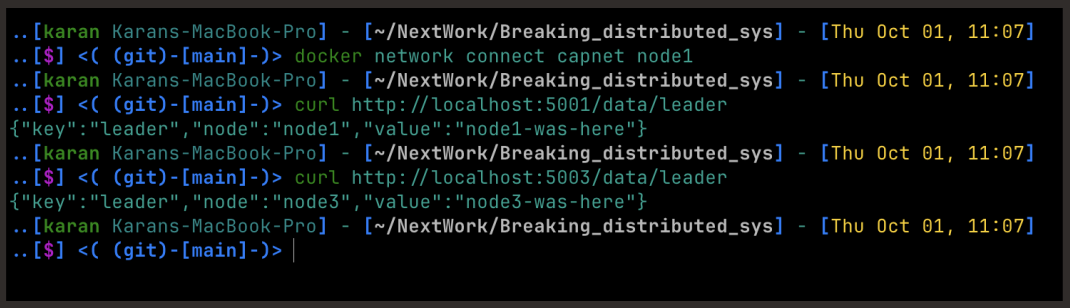
BREAK A DISTRIBUTED SYSTEM ON PURPOSE

By Karan | October 2026



Project Overview: Exploring the CAP Theorem

Goals and motivation

We're gonna build a distributed 3 node key-value store system, using python, flask and docker, which we will then use to create empirical evidenc for the workings of the CAP theorem by delibrately creating a partition in the network in watching the decision between C (consistency) VS A(availability).

Through this we want to gain a clear mental model of CP vs AP systems and better our understanding of their real-world application in software like PostgreSQL and Cassandra

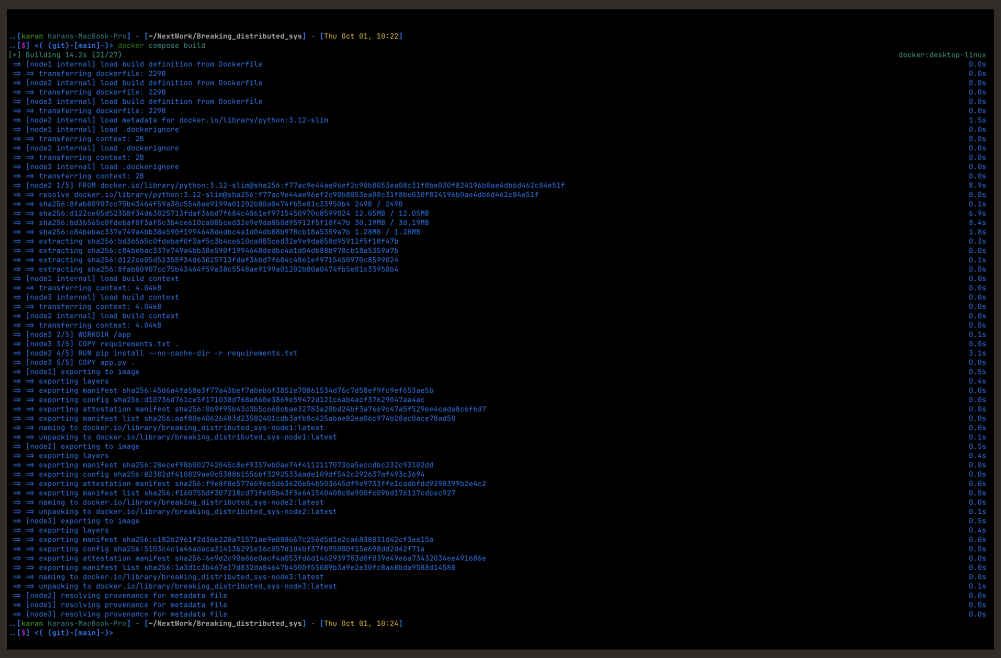
Finally we'll add a timestamp based conflict resolution mechanism so that nodes have a way of merging diverged data once the partition has healed.

Building the Distributed Node and Cluster

Setting up the project

We're writing the flask application that will act as a distributed key_value store node. Then creating a docker file, along with the requirements file and the Docket Compose configuration.

After that we will build the docker image to make sure that it can compile without errors.



Project files and their roles

The four files we should now have are:
app.py : containing the core logic for the distributed system and how they should function in each mode.

requirements.txt: containing the dependencies for the application and their version number.

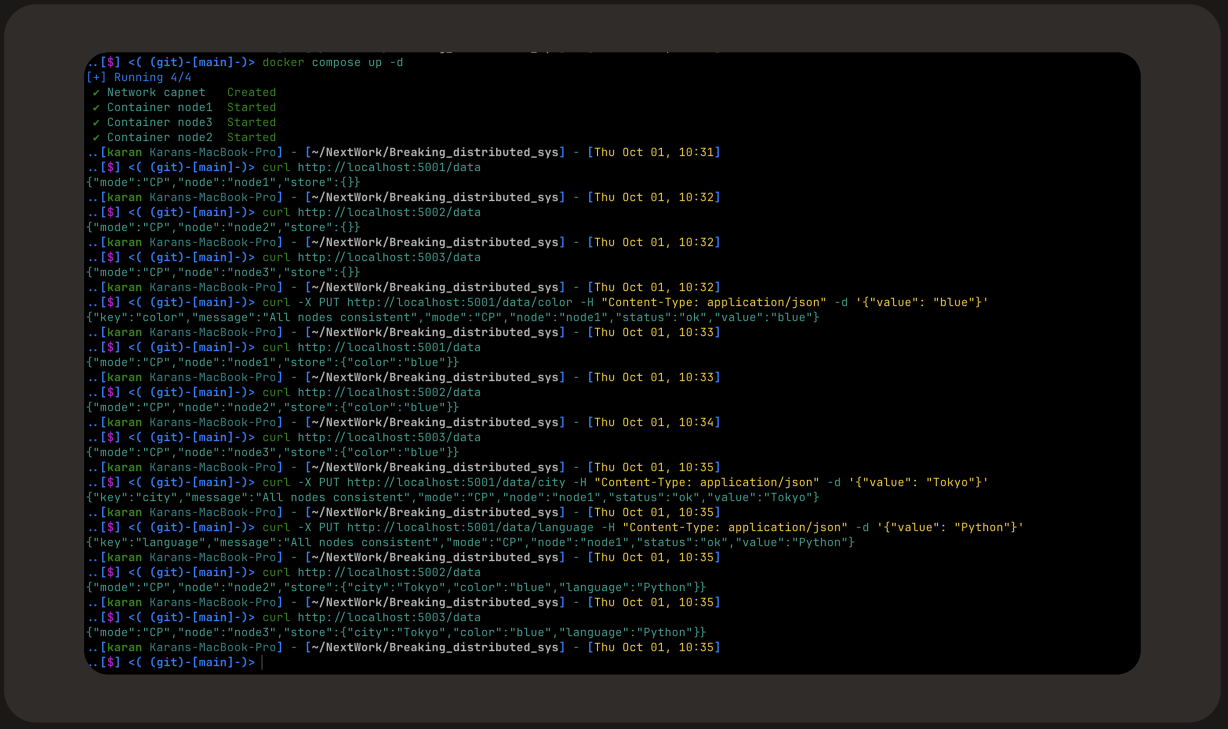
Dockerfile: containing the exact sequence of steps docker should execute.

docker_compose.yml: instructs how each container should inherit data about the application (app.py) along with the environment variables that will bind to the system, like port 5001, 5002 and 5003 mapping to port 5000 on the machine.

Launching the Cluster and Verifying Replication

Normal operation goals

Now we're ready to make the network live and bring up our 3 node cluster. Once live we can attempt to write data to one node to see the repliaction process in execution and verify that the cluster is able to operate as a unified system.



Consistency before any partition

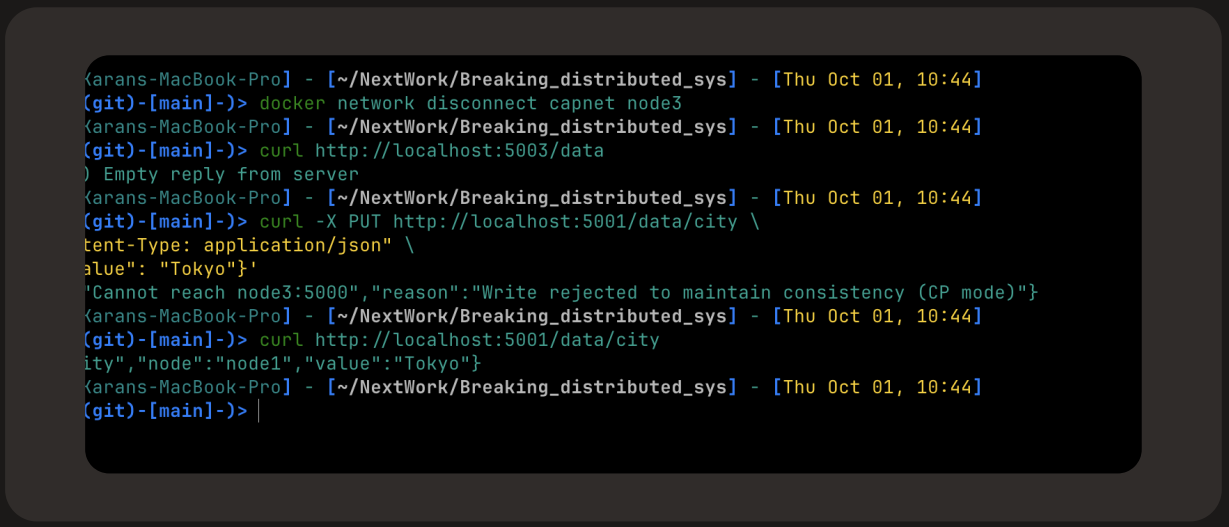
In CP mode, writes to one node aren't saved locally until the other nodes are able to confirm that they have recieved the data through the replicate method we have defined in our app logic. This means a write to node1 will result in the new data being sent to the other nodes through the replicate end point and once all n-nodes respond with success, will the new data be written locally.

Here the CAP theorem is not applicable because the core trade-off between the choosing consistency or availability will no occur as long as the network is intact. Once a partition occurs in the network is when the theorem becomes applicable.

Simulating a Network Partition in CP Mode

Partition simulation approach

To see how a chosen mode (CP or AP) affect the functionality of the application and its behaviour, we can now simulate "breaking" the network we have made by taking one of the nodes down and observing it's impacts on writes to the cluster, also allowing us to then test what happens to the data when the node comes back online.



Why CP mode rejects writes

We can see here that removing node 3 from the cluster prevents writes to other nodes in the cluster, and within the error output we can see that the "Write rejected to maintain consistency (CP mode)".

Since the cluster is operating with consistency as the priority, all writes are rejected until it can be verified that all nodes are able to receive and write to their own local stores.

Observing Split-Brain in AP Mode

Switching to AP mode

Now we can try to run the cluster in the Availability mode, and see how the cluster is able to handle writes when all of the nodes are not online to process the new information. We can predict that this will result in different nodes having different data saved locally for the same key.

CP vs AP trade-offs explained

The tradeoff we can see between the two modes here:
CP (Consistency): writes to the cluster are prevented until we can verify ever node is reachable and able to handle processing new data.

AP (Availability): writes are stored first and replicated later, which means that in the event a node is offline, a copy of the data will still be retained in the node that was written to, providing some form of preservation.

Implementing Conflict Resolution with Last-Write-Wins



Trade-offs of eventual consistency

The tradeoff with the last-write-wins startegy is that the data discarded could be perfectly valid data but older than other recent data and could hence get discarded.

Reflections and Key Takeaways

Tools and concepts mastered

Key concepts/tools learnt:
- CAP theorem
- Distributed systems
- Docker
- Home labbing.

Time and challenges

This project took me an hour and a half of solid work behind the terminal and text editor, since I replicated the code by hand to understand what was being done every step of the way.

Looking ahead

I did this project to get an intro to docker and learn a thing or two about distributed systems and this has been incredibly useful in those domains. I'd like to learn more about distributed systems and how modern systems are archetected.